Smart System Monitoring: Real-Time Detection of Workload Modes in Computer Systems Using Machine Learning Methods

Kamil Maciejko

Contents

Summary 1
Goal 2
Project concept 2
Data collection 3
Data Preprocessing and EDA 5
Model development 7
Real-Time Prediction 9
Discussion and limitations 11
Conclusion 12

Summary

This paper investigates the use of machine learning methods to monitor and classify laptop performance modes in real time. By analysing hardware performance metrics such as CPU and GPU utilisation, a data-driven system was developed to categorize workloads into three operational states: low, standard, and high performance. Several machine learning methods, including decision tree–based models and artificial neural networks, were evaluated, with the XGBoost model achieving the highest classification accuracy.

Beyond model development, the study also includes a practical simulation demonstrating how such predictions can support dynamic adjustment of system performance settings during active device usage. Although the results are based on data collected from a specific hardware configuration, they provide a strong proof of concept for automated and intelligent resource management in IoT-related environments.

Goal

The main purpose of this paper is to explore an IoT-related topic from data science perspective and give insightful thoughts about usefulness of machine learning in this area. The project aims to provide and evaluate a data-driven system for classifying laptop workload states based on real-time metrics (CPU, GPU, memory, disk, or network usage). Using collected data, a ML model will be trained that predicts a workload category (low, standard, or high-performance). Finally, a practical solution for the model will be developed that will acquire system workload data in real-time. Application will dynamically provide a recommended performance mode based on model's prediction.

Project concept

Data will be collected using real-time monitoring of system's performance metrics. Final training dataset will be used to build various models using machine learning methods. The best model will be tested in real time on new, unlabeled data to verify its usefulness in practice.

Model should be able to define three workload states based on a provided data:

- Low workload, silent, blue mode

Represents low system utilization characterized by low CPU, GPU or memory activity. In this state, performance limits could be reduced to minimize power consumption, temperature and fan noise.

- Standard workload, white mode

Represents moderate and daily usage with balanced resource usage. The system should maintain adaptive performance to handle occasional workload spikes without unnecessary power increase.

- High performance, red mode

Represents high workload, particularly high CPU and/or GPU utilization, increased memory usage, and higher temperatures. In this state, maximum performance limits should be enabled to maintain best performance under demanding tasks.

Data collection

Training data will be obtained through real-time monitoring of laptop's system performance based on selected metrics. They will be acquired during predefined workload sessions that will simulate corresponding target classification labels.

Each session will last 20 minutes, and will represent one of the following workload types:

Blue mode session – Low activity. Example activities: idle system, reading or writing in a text editor, watching a video, browsing internet, simple coding tasks.

White mode session – Moderate activity. Example activities: multiple easy tasks e.g.: watching videos while coding, chatting with LLMs, multiple browser tabs opened, few applications running at once, light downloads.

Red mode session – High activity. Example activities: gaming, heavy downloads, intense utilization of multiple applications, various processes running simultaneously.

Sessions will be conducted separately, with an appropriate time space between them. They will be repeated with different activities in order to improve dataset variability. In-built system performance mode will be set to standard/automatic and will remain fixed. These actions will minimize bias and ensure quality. Data obtained during these sessions will be adequately labeled and later processed.

Performance metrics will be collected every second. The following features will be recorded:

- CPU Usage (% per core, 16 cores) and CPU Speed (MHz)
- GPU Usage (%) and GPU Temperature (^{o}C)
- RAM Usage (%)
- Disk read bytes (bytes) and Disk write bytes (bytes)
- Network bytes sent (bytes) and Network bytes received (bytes)

Data will be collected using a laptop Lenovo Legion 5 Pro Gen 7 (16'' AMD). CPU is AMD Ryzen 7 6800H, GPU is NVIDIA GeForce RTX 3070 with 16GB of RAM.

It is important to mention that an external monitor will be connected and used to display ongoing data collection process. It is important to supervise inflow of metrics to ensure uninterrupted and swift operation. It may impact laptop's workload, however influence on classification is considered negligible as it is low and equal across all sessions.

Full plan of sessions with activities presents Table 1. Schedule has been developed to simulate real workload of a system.

Table 1. Plan of data collection sessions with activities and planned time. *(Own work)*

	Blue Session	White Session	Red Session
Turn 1	0-5min: Idle (screen on) 5-10min: PDF reading + text editor 10-15min: Simple Python coding 15-20min: YouTube video	0-5min: PowerPoint + few tabs opened + occasional image download 5-10min: YouTube video + LLM interaction + Python coding 10-20min: Downloading and installing small software + Spotify in the background	0-10min: Online, demanding game + discord call + few applications idle in the background 10-20min: Continue previous operations + start a download
Turn 2	0-5min: Web Browsing 5-10min: Spotify, changing music and browsing actively 10-15min: PDF reading, Spotify in the background 15-20min: Idle (screen on)	0-10min: Offline, light game 10-15min: Browsing Social Media posts and videos + few tabs opened 15-20min: Text editor + generating and downloading AI images + YouTube video	0-20min: Heavy Python coding + continuous interaction with LLM + multiple tabs opened + YouTube video + Spotify + download in the background

At least 10min space between each session has been deemed sufficient for system to cool down after activities. Performance monitoring will be turned on only when activities will already start. These measures will prevent bias which combined with variety of operations should ensure quality of data and good generalisation of the model. In total, around 1200 records per session should be obtained, resulting in 7200 rows for training data.

Data Preprocessing and EDA

Couple adjustments had to be made to data to prepare a final dataset that will be fit for training of machine learning models. First of all, an average CPU usage across cores was computed. As for disk and network metrics, they are special as raw data for them show cumulative values which in this form are utterly useless, however particular transformations could turn them into informative features. Delta (or change) has been calculated to represent dynamics of these variables and raw ones were deleted from the dataset. Computer systems are characterized by often spikes during e.g. opening apps etc. and in order to mitigate their influence and ensure consistency across time, rolling features have been introduced. For every variable (except individual CPU core usage) a mean and standard deviation for previous 10 records have been computed.

Final training dataset after mentioned changes consists of 7200 records with 43 explanatory features and dependent variable with 3 equally distributed classes.

Data have been collected during experimental sessions with predefined activities in order to achieve the clearest distinction possible. Such approach resulted in visible differences between groups. In particular, red mode distinguishes clearly from the others which can be seen in Table 2 and on Figure 1 and 2. Average usage of CPU and GPU for the red mode reaches 55 and 61% which is many times higher than for the other groups. Also, temperature of GPU rises drastically to over 77°C on average. RAM usage doubles in comparison to other modes. What can be additionally seen on figure 2, presenting distribution of selected variables, GPU metrics for the red mode are divided into two visible clusters. Extremely high values are most probably a result of gaming activities that burden graphic card greatly, incomparably with any other activity. Even though blue and white mode metrics are visibly smaller, they can be clearly distinguished between each other. Mean CPU and GPU usage for the standard mode are almost twice higher than for the silent one. GPU temperature and RAM usage are also significantly different.

Table 2. Comparison of means for selected features by performance mode. *(Own work)*

Label/Feature	**cpu_usage_avg**	**gpu_usage**	**gpu_temperature**	**ram_usage**
Blue	5.23	8.06	51.20	41.53
White	9.27	19.11	56.15	48.55
Red	54.92	61.36	77.02	83.51

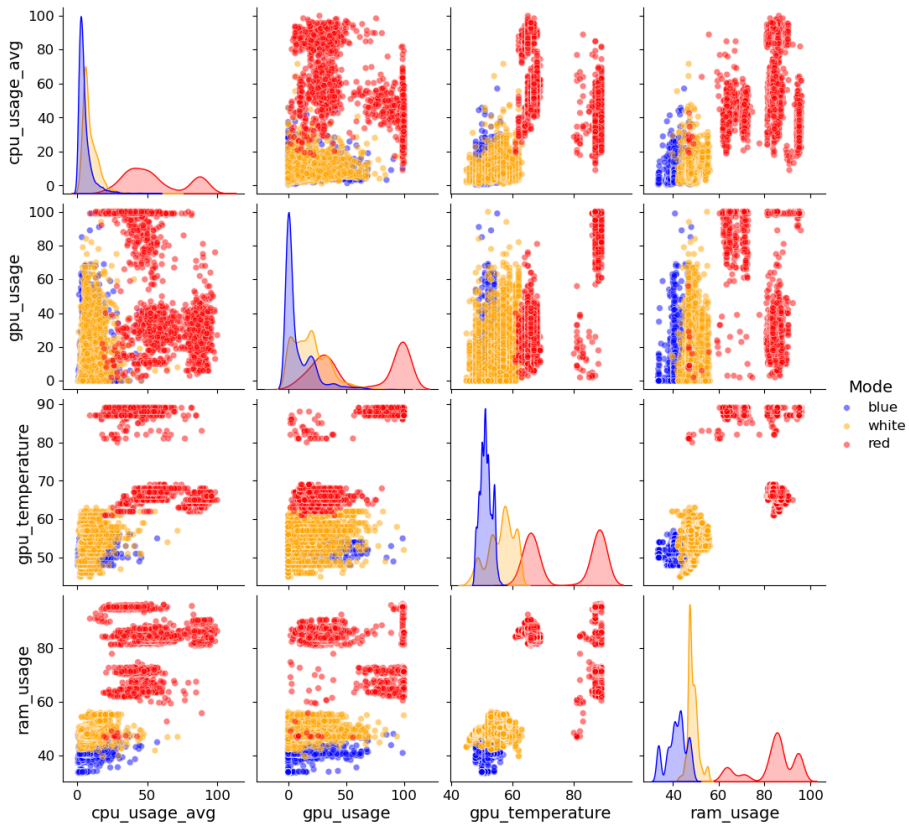

Fig.1. Comparison of selected features by performance mode. *(Own work)*

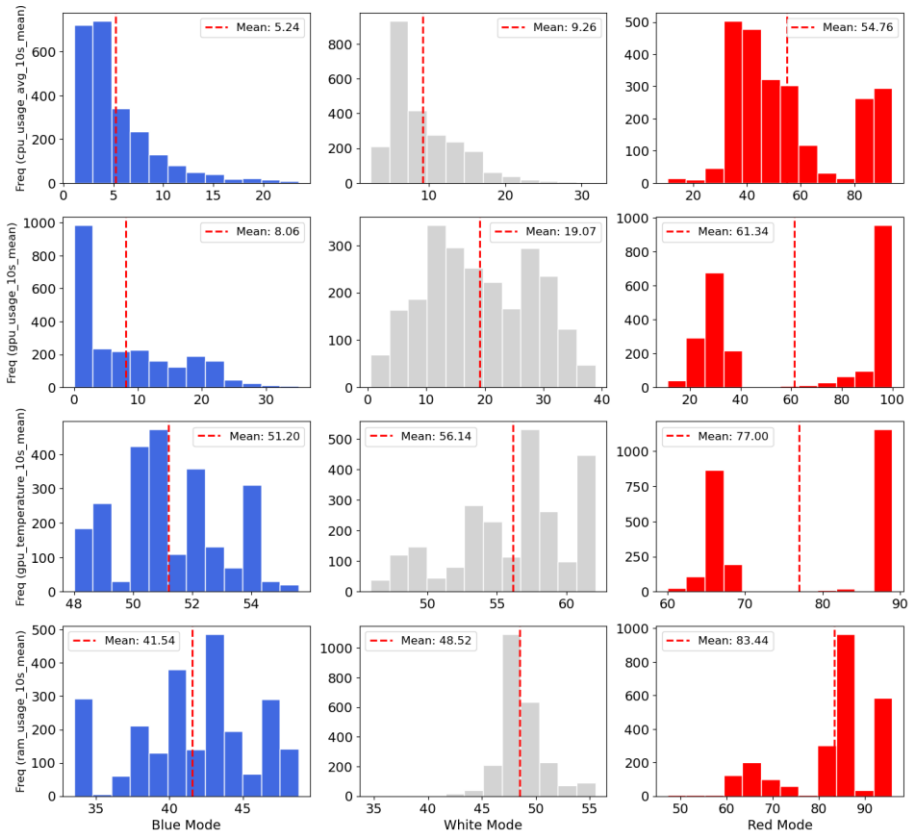

Fig.2. Distribution of selected features by performance mode. *(Own work)*

ANOVA test conducted for all features for classifiers showed that differences between groups are statistically significant for every one of them (for alpha = 0.05) except for disk and network variables. For them we cannot reject a null hypothesis.

After data preparation a model development is finally possible.

Model development

To find the best model, capable of classifying performance modes, a series of machine learning methods have been used: decision trees, bagging, random forest, xg boost and artificial neural network. Size of test sample was set to 20%. Models have been tested for different values of hyperparameters using cross-validation method. To ensure correct learning process for ANN, data has been standardized and logarithmic transformation has been conducted on features with skewness exceeding 2.

- For decision trees models with 5 and 20 branches have been trained.
- Bagging was tested on a number of estimators from a list (5, 8, 10, 20, 25, 30, 40, 50, 75, 100) with the best result with 75 estimators.
- Random forest method was tested for number of estimators in (10, 20, 30, 40, 50, 100) and maximal depth of a decision tree in (2, 3, 4, 5, 8, 10, 15, 20, 30) with best results for 100 and 15 respectively.
- XG Boost method was tested for a number of estimators in (50, 75, 100, 150), learning rate in (0.01, 0.05, 0.1) and maximum depth in (2, 4, 6, 8, 10, 12) with best results for 150, 0.1 and 4 respectively.
- Artificial Neural Network activation function was set to RELU, optimizer to Adam and batch size to 32. Different configurations of hidden layers were tested in a pattern (neurons in a layer, number of hidden layers). The configurations were: [(10, 1), (20, 1), (10, 2), (20, 2), (40, 2), (10, 3), (20, 3), (40, 3), (20, 4), (20, 5), (64, 2), (64, 3)] with best results for (40, 2) network. Early stopping methods showed the best results for number of epochs = 14.

Results of training appeared to be exceptionally good, and models are extremely accurate in correct classification of performance mode for a laptop. It is shown in Table 3.

Table 3. Accuracy of trained models. *(Own work)*

Model	Accuracy
Decicion Tree (5)	0.965
Decision Tree (20)	0.988
Bagging	0.996
Random Forest	0.999
XG Boost	1.000
ANN	0.976

The strong performance of the evaluated models can be attributed to the high quality of the training dataset. The data were collected under controlled experimental conditions, resulting in clearly separable workload classes. Additionally, all features are numerical, which is particularly suitable for decision tree–based methods and may partially explain the comparatively lower performance of the artificial neural network model.

As a result, most models achieved very high predictive accuracy and produced only a small number of classification errors. Notably, the XGBoost model correctly classified every observation in the test dataset, achieving an accuracy of 100%. Given this performance, it was selected as the final model for real-time workload mode prediction. A detailed summary of its performance, including the confusion matrix, is presented in Figure 3.

While these results are highly encouraging, they may also reflect the relatively well-separated nature of the experimental dataset. Such conditions can raise concerns regarding overfitting or limited generalization capability. However, the results remain consistently high across the training and test datasets as well as in cross-validation experiments, which suggests that the strong performance is primarily caused by the clear separability of the workload classes. This interpretation is further supported by the similarly high performance observed across multiple models.

```
Best parameters: {'learning_rate': 0.1, 'max_depth': 4, 'n_estimators': 150}
Best CV accuracy: 0.9987847222222224
Test Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

        blue       1.00      1.00      1.00       480
         red       1.00      1.00      1.00       480
       white       1.00      1.00      1.00       480

    accuracy                           1.00      1440
   macro avg       1.00      1.00      1.00      1440
weighted avg       1.00      1.00      1.00      1440


Confusion Matrix:
 [[480   0   0]
 [  0 480   0]
 [  0   0 480]]
```

Fig.3. Parameters, performance and confusion matrix for the best model (XG Boost) *(Own work)*

As mentioned previously, rolling metrics were introduced to mitigate the influence of outliers and improve the model's learning capability. To assess their actual impact, an additional experiment was conducted in which the same XGBoost model was trained on a dataset without rolling features. The resulting model achieved an accuracy of 0.994, misclassifying nine observations in total. These results suggest that including rolling statistics slightly improves model performance; however, even without them the model is still able to classify the data with very high accuracy.

Real-Time Prediction

The final model showed very promising results during the testing phase; however, its real value must be verified in a practical scenario. Internet of Things (IoT) systems rely on a continuous stream of real-time data that must be analysed instantly in order for devices to respond appropriately to changing environmental conditions. To address these requirements, a practical solution incorporating real-time data processing and prediction has been proposed.

The model was integrated into a simulation function that consists of four main stages. In the first one, real-time data describing system performance are collected in a similar way to the data collection procedure described earlier in this project. Next, the raw metrics are processed by a dedicated subfunction that transforms them into an analysable format. Once the data are prepared, the trained model is applied to classify each observation into one of the predefined workload modes (blue, white, or red) based on the provided system metrics. Finally, each record along with its predicted class is stored, and a visualization displaying all previously collected observations is generated.

To enhance monitoring capabilities and better illustrate the real-time classification process, the visualization is updated every five seconds. The function returns a complete data frame containing all collected observations and their corresponding predictions, allowing for further analysis.

It is crucial to emphasize that this solution is an experimental function aimed at testing a model in closed environment and is not universally applicable. Data can be collected only on a device similar to the one used in the data collection procedure i.e. with 16 CPU cores, Nvidia GPU etc. Displaying real-time updating plot and checkpoint information have a purpose of better visualisation and supervision of the process rather than optimization. Therefore, the code can impact workload more significantly than an initial function for a pure monitoring of system's performance. The application works most efficiently with models that don't require standardization and logarithmic transformation of data like decision trees, bagging, xgboost etc. It is possible to use models like ANN that need those changes but requires an implementation of additional parameters.

In order to test a real-time application of the model using this function, a 30-minute session consisting of various activities has been conducted. To reduce influence of an initial function call, a 10 second window was added. Contradictory to a previous strategy, no time space between activities is planned. This approach is a better simulation of a real scenario that tests how model handles sudden changes of workload and how predictions shape during breaking points.

Plan of the session is presented in a Table 4.

Table 4. Activity plan for the real-time prediction test with expected modes for particular time frames. *(Own work)*

Activity	PDF Reading	Demanding Game	YouTube + LLM	Coding + Spotify + Browser	Idle with Screen On	Heavy Python Coding + YouTube + Browser
Time	0-5	5-10	10-15	15-20	20-25	25-30
Expected mode	Blue	Red	White	White	Blue	Red

Results of the experiment in a form of timeline with predictions are shown on Figure 4.

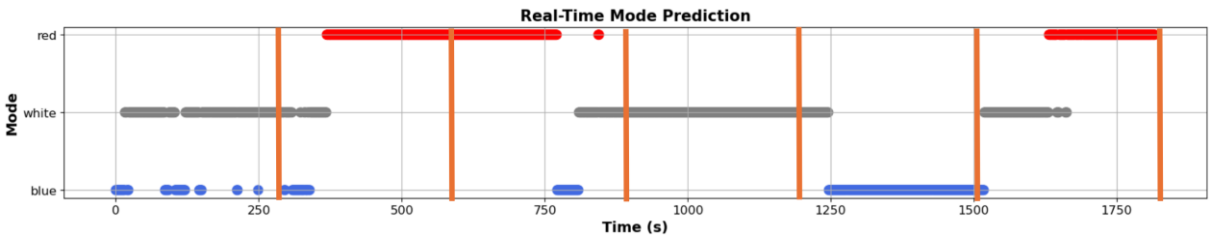

Fig.4. Real-time predictions of model. Orange lines show an orientational time of an activity switch. *(Own work)*

The model predicted the low workload mode 378 times, the standard workload mode 841 times, and the high workload mode 581 times. The results indicate that during the initial phase of the simulation the workload was higher, which is likely related to the impact of running the monitoring and prediction code itself as simple activities, such as reading a PDF document, were predominantly classified as the standard workload mode. It was also observed that transitions between predicted workload modes occur gradually rather than instantaneously. Unlike the data collection procedure, where time gaps between workload sessions were introduced, the simulation involved immediate switching between activities, which influenced the resulting predictions.

This behaviour does not indicate incorrect model performance; rather, it reflects the natural dynamics of system operation. After periods of intensive activity, the system requires time to return to a lower workload state, while increased activity similarly requires time to raise the overall system load.

These observations provide an interesting perspective on how such predictions could be utilized in practice, for example in the automatic adjustment of a laptop's performance mode based on the current workload conditions.

Discussion and limitations

In order to fully understand project's scope and context, several important considerations and limitations have to be emphasised. First and foremost, the study was conducted as an experiment in a controlled, predefined environment. The dataset was collected in a structured process that consisted of selected activities simulating realistic device usage. While this approach ensured clear separation between workload classes, it also means that the collected data reflect conditions designed for experimental purposes rather than completely unconstrained real-world usage.

Additionally, the data were collected using a single laptop with specific performance capabilities and hardware characteristics. As a result, the trained model captures workload patterns that are partially dependent on this particular system configuration. When applied to data collected from devices with different components or characteristics, the predictive accuracy of the model may decrease.

Although the training dataset can be easily expanded and diversified, its current size remains relatively limited and is based on a predefined set of activities that were considered sufficient for the purposes of the experiment. Real-world usage involving a broader range of tasks may generate different workload patterns that were not represented in the training data and could therefore lead to occasional misclassification.

The real-time prediction function should also be considered an experimental prototype designed to imitate practical application. Insights and conclusions from the results are important and valuable but it is not an optimized, ready-to-use code in a production environment. While the implementation demonstrates how real-time monitoring and classification could operate in practice, it is not intended to represent a fully optimized, production-ready system.

Despite these limitations, the project demonstrates that system workload states can be effectively identified using machine learning techniques based solely on real-time performance metrics. The proposed analytical pipeline, from data collection and feature engineering to model training and real-time application, provides a proof of concept for how such models could be integrated into monitoring systems or IoT-oriented solutions. The results therefore highlight the potential of machine learning methods for supporting intelligent system monitoring and automated performance management.

Conclusion

The goals of this project were successfully achieved. The data collection process provided a high-quality training database with 3 clearly distinguishable groups. The collected data were transformed and extended with additional features, resulting in a more informative representation of system performance metrics. They were then used to train several machine learning models. The quality of the dataset is reflected in the very strong performance of the evaluated methods, all of which achieved classification accuracies above 96%. Among them, XGBoost method demonstrated the best performance and correctly classified every record of the test dataset. It has been therefore selected as the final model.

To further evaluate the practical applicability of the proposed approach, a simulation of a real-world scenario was conducted. The selected model was integrated into a function that continuously monitored system performance and transformed the collected metrics into an analysable format. Based on this information, the model provided real-time predictions of the laptop's workload mode for each observation.

The results of this experiment also proved promising and provided several insights into the dynamics of system workload. The proposed approach therefore demonstrates potential for practical applications, such as the automatic adjustment of device performance modes based on real-time workload predictions. Future research could focus on extending the dataset with a wider range of device types and usage scenarios, as well as further optimizing the real-time monitoring framework for deployment in practical IoT environments.